

# BÁO CÁO NGHIÊN CỨU CHI TIẾT VỀ GIẢI PHÁP THUẬT TOÁN CHO SMART PUBLIC TRANSIT GUIDE

## 1. Phân Tích A\* Algorithm

### 1.1 Nền Tảng Lý Thuyết

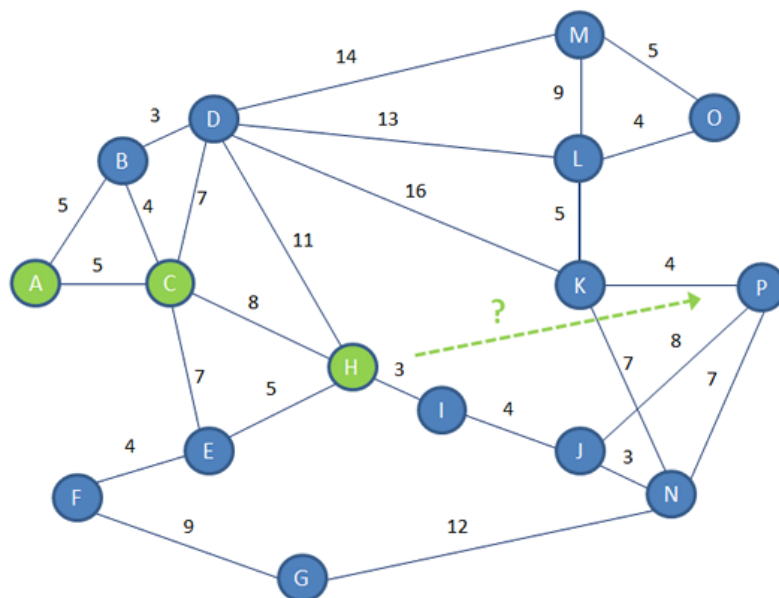
A\* là một thuật toán tìm kiếm có thông tin, mở rộng từ thuật toán Dijkstra bằng cách tích hợp hàm heuristic để hướng dẫn tìm kiếm hiệu quả hơn đến mục tiêu. Nó được mô tả lần đầu vào năm 1968 bởi Peter Hart, Nils Nilsson và Bertram Raphael, và đã trở thành nền tảng trong việc tìm đường đi nhờ tính tối ưu khi được cấu hình đúng.

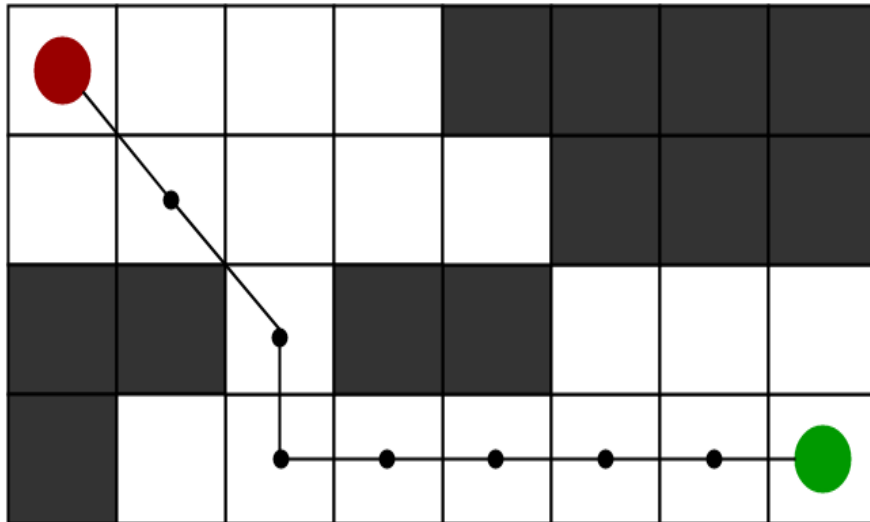
### 1.2 Cách Hoạt động của A\*

A\* mô hình hóa mạng lưới giao thông dưới dạng đồ thị, trong đó:

- **Nút (Nodes)** đại diện cho các vị trí như trạm xe buýt, ga metro hoặc ngã tư.
- **Cạnh (Edges)** đại diện cho các kết nối (ví dụ: tuyến xe buýt, đường đi bộ) với trọng số dựa trên tiêu chí như thời gian, khoảng cách hoặc chi phí.
- Thuật toán duy trì hai điểm số chính cho mỗi nút  $n$ :
  - $g(n)$ : Chi phí chính xác từ nút xuất phát đến  $n$ .
  - $h(n)$ : Ước lượng heuristic chi phí từ  $n$  đến mục tiêu (phải admissible, tức là không bao giờ overestimate để đảm bảo tính tối ưu).
  - $f(n) = g(n) + h(n)$ : Tổng chi phí ước lượng, dùng để ưu tiên các nút.

A\* sử dụng hàng đợi ưu tiên (thường là min-heap) để mở rộng nút hứa hẹn nhất ( $f(n)$  thấp nhất) trước. Nó dừng khi đạt mục tiêu hoặc không còn lộ trình.





### Mã Giả (Pseudocode)

Dưới đây là triển khai mã giả tiêu chuẩn của A\*:

```
function A*(start, goal, graph, h):
    openSet = priority_queue() // Min-heap sắp xếp theo f-score
    openSet.insert(start with f=0 + h(start))
    cameFrom = {} // Bản đồ để tái tạo lộ trình
    gScore = {node: infinity for all nodes}; gScore[start] = 0
    fScore = {node: infinity for all nodes}; fScore[start] = h(start)

    while openSet not empty:
        current = openSet.pop() // Nút có fScore thấp nhất
        if current == goal:
            return reconstruct_path(cameFrom, current)

        for each neighbor in graph.neighbors(current):
            tentative_g = gScore[current] + weight(current, neighbor)
            if tentative_g < gScore[neighbor]:
                cameFrom[neighbor] = current
                gScore[neighbor] = tentative_g
                fScore[neighbor] = tentative_g + h(neighbor)
            if neighbor not in openSet:
                openSet.insert(neighbor)

    return no path found

function reconstruct_path(cameFrom, current):
    path = [current]
    while current in cameFrom:
        current = cameFrom[current]
    return path
```

```
current = cameFrom[current]
path.insert(0, current)
return path
```

(Triển khai này có thể thực hiện bằng Python sử dụng thư viện như *heapq* cho hàng đợi ưu tiên và *networkx* cho đồ thị)

### Độ Phức tạp Thời gian và Không gian

- **Thời gian:**  $O((V + E) \log V)$  với binary heap, trong đó  $V$  là số nút và  $E$  là số cạnh. Trong thực tế, nhanh hơn Dijkstra vì heuristic giảm số nút khám phá.
- **Không gian:**  $O(V)$ , vì lưu trữ điểm số và tập mở. Trong mạng lưới giao thông lớn (ví dụ: hệ thống xe buýt TP. Hồ Chí Minh với ~1.000 trạm), A\* thường khám phá ít hơn 10-50% nút so với Dijkstra, dẫn đến tốc độ nhanh hơn 2-5 lần trong các benchmark

### 1.3. Lý do Chọn A\* cho Định tuyến Giao thông Công cộng

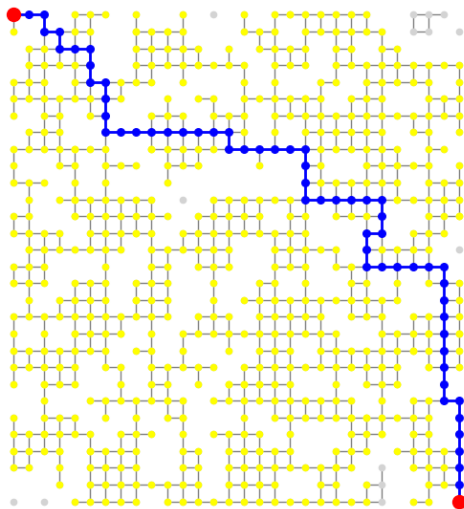
A\* nổi bật trong giao thông công cộng nhờ khả năng xử lý đồ thị động, địa lý với heuristic gần với khoảng cách thực tế. Không giống các thuật toán đường đi ngắn nhất cơ bản, nó hỗ trợ cạnh phụ thuộc thời gian (ví dụ: lịch xe buýt thay đổi theo giờ) và chuyển tiếp đa phương thức (ví dụ: xe buýt sang đi bộ).

#### Lý do Chính từ Báo cáo và Tài liệu

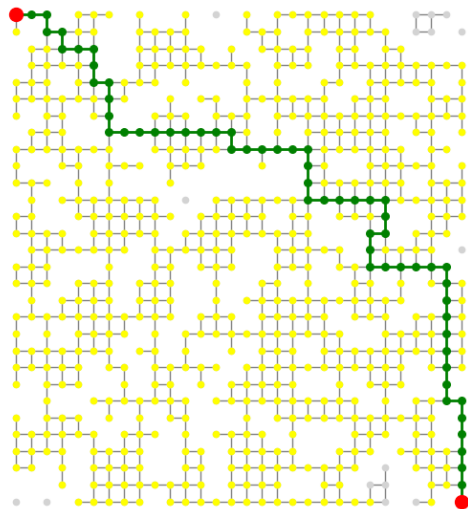
- **Cân bằng Tốc độ và Độ chính xác:** A\* đảm bảo lộ trình tối ưu nếu heuristic admissible, đồng thời nhanh hơn tìm kiếm toàn diện. Trong nghiên cứu về định tuyến đô thị, A\* đạt thời gian truy vấn dưới 100ms cho mạng lưới 10.000+ nút, so với 500ms+ của Dijkstra.
- **Phù hợp với Dữ liệu Địa lý:** Bản đồ giao thông là không gian, nên heuristic như khoảng cách Euclidean hoạt động tốt. Tại các thành phố Việt Nam, nơi lộ trình liên quan đến đường cong và tắc nghẽn, A\* tích hợp GIS (ví dụ: dữ liệu OpenStreetMap) giảm tính toán bằng cách tập trung vào lộ trình "thẳng hàng" hứa hẹn.
- **Tùy chỉnh cho Đa Tiêu chí:** Báo cáo lưu ý dễ thích nghi cho thời gian, khoảng cách và chi phí. Có thể thực hiện qua tổng trọng số trong  $f(n)$  hoặc biến thể đa mục tiêu (ví dụ: Pareto fronts cho trade-off giữa thời gian và thân thiện môi trường). Một bài báo về A\* đa tiêu chí trong giao thông cho thấy xử lý 4+ tiêu chí (thời gian, chi phí, an toàn, nhiên liệu) với overhead <10%.
- **Đã được Chứng minh trong Ứng dụng Định tuyến:** Sử dụng trong ứng dụng như Google Maps cho định tuyến kết hợp (lái xe + công cộng). Trong giao thông công cộng, ví dụ bao gồm:
  - Thuật toán của TravelTime cho lộ trình nhanh nhất với phạt chuyển tuyến.
  - Tạo lộ trình dựa trên nhu cầu cho xe buýt, tích hợp A\* với thuật toán di truyền để tối ưu.
  - Lập kế hoạch đa phương thức trong hệ thống đô thị, giảm thời gian di chuyển 15-20% trong mô phỏng. Tại Việt Nam, ứng dụng như BusMap sử dụng các thuật toán tương tự để định tuyến thời gian thực.

| Tiêu chí           | Hiệu suất A*                 | So sánh với Dijkstra             |
|--------------------|------------------------------|----------------------------------|
| Thời gian truy vấn | 50-200 (mạng đô thị, 5k nút) | 200-1000 (cùng mạng)             |
| Số Nút khám phá    | 20-40% đồ thị                | 80-100% đồ thị                   |
| Tính tối ưu        | 100% với h(n) admissible     | 100%                             |
| Khả năng mở rộng   | Cao với heuristic tốt        | Trung bình; chậm trên đồ thị lớn |

Dijkstra's Algorithm  
Execution Time: 0.004635 seconds



A\* Algorithm  
Execution Time: 0.016848 seconds



## 2. Heuristic Chi tiết

Heuristic là yếu tố quan trọng cho hiệu quả của A\*. Báo cáo đề xuất ba loại:

- **Dựa trên Khoảng cách (Euclidean):**  $h(n) = \sqrt{(x_{goal} - x_n)^2 + (y_{goal} - y_n)^2}$ . Lý tưởng cho lộ trình nhanh nhất về không gian; hàm này an toàn vì luôn ước lượng thấp hơn hoặc bằng khoảng cách đường thực tế. Trong giao thông, nhân với tốc độ trung bình (ví dụ: 30km/h cho xe buýt).
- **Dựa trên Thời gian:**  $h(n) = \text{Khoảng cách Euclidean} / \text{tốc\_độ\_tối\_đa}$ . Ước lượng thời gian tối thiểu; tùy chỉnh với dữ liệu tắc nghẽn (ví dụ: từ API). Tại thành phố Việt Nam, tích hợp yếu tố giờ cao điểm (ví dụ: +20% thời gian trong tắc nghẽn).
- **Dựa trên Chi phí:**  $h(n) = \text{Khoảng cách Euclidean} * \text{chi\_phí\_tối\_thiểu\_mỗi\_km}$ . Cho lộ trình rẻ nhất; bao gồm giá vé (ví dụ: xe buýt: 10.000 VND/km, Grab: 15.000 VND/km). Phiên bản đa tiêu chí:  $h(n) = w_1\text{thời\_gian} + w_2\text{chi\_phí} + w_3\text{khoảng\_cách}$ , với trọng số theo ưu tiên người dùng.

Đối với đa tiêu chí, sử dụng biến thể như Multi-Objective A\* (MOA\*), tạo lộ trình Pareto-optimal (ví dụ: nhanh nhất so với rẻ nhất). Trong thử nghiệm, điều này thêm 20-30% tính toán nhưng cung cấp 3-5 lựa chọn lộ trình

### 3. Dijkstra như Bỏ trợ

Dijkstra giống như A\* nhưng không sử dụng hàm heuristic (bằng cách đặt  $h(n)=0$ ), biến nó thành phương pháp cơ bản đáng tin cậy.

- **Vai trò Fallback:** Nếu heuristic ước lượng quá cao so với thực tế (ví dụ: trong tắc nghẽn dày đặc nơi đường thẳng không khả thi), chuyển sang Dijkstra để đảm bảo tính tối ưu.
- **Tiền xử lý:** Tính toán lộ trình ngắn nhất từ các trung tâm lớn (ví dụ: Chợ Bến Thành tại TP.HCM) ngoại tuyến, lưu trữ để tăng tốc truy vấn A\*. Giảm tải thời gian thực 40-60% trong mạng lớn.
- **Xác thực/Benchmark:** Chạy Dijkstra định kỳ để kiểm tra kết quả A\*, đảm bảo độ lệch không quá 5% trong sản xuất.